

ALCOBATON (90)

GF

Arquitectura de Computadoras
Apellido: _____

Parcial 2- Fecha 1 - 15/11/2024
Nombre: _____

TEMA B
DNI: _____

1. Indicar cuantos atascos se producen del tipo RAW y BTS ejecutando el programa, BTB y Delay Slot están deshabilitados (40 pts)

.data		loop:	ld	\$t2, VEC1(\$t0)
VEC1: .word 5, 10			daddi	\$t1, \$t1, -1
VEC2: .word 0			sd	\$t2, VEC2(\$t0)
.code			daddi	\$t0, \$t0, 8
daddi \$t0, \$0, 0			bnez	\$t1, loop
daddi \$t1, \$0, 2			halt	

Ejercicio	Puntos
1	40
2	15
3	5
4	30
Total	90

Con Forwarding deshabilitado: RAW: 3 (5 pts) BTS: 1 (5 pts) CPI: 21/13 (10 pts)
Con Forwarding habilitado: RAW: 0 (5 pts) BTS: 1 (5 pts) CPI: 18/13 (10 pts)

2. El siguiente programa recorre un arreglo de números (TABLA1) y genera un segundo arreglo (TABLA2) con los números que sean impares. Además guarda en RES la cantidad de elementos guardados en el segundo arreglo. Completar instrucciones faltantes. (5 pts c/respuesta)

.data		LOOP:	ld \$t4, TABLA1(\$t2)
TABLA1: .word 11, 2, 6, 7, 4, 21, 10, 4			andi \$t5, \$t4, 1
CANT: .word 8			beqz \$t5, FUERA
RES: .word 0			daddi \$t1, \$t1, 1
TABLA2: .word 0			sd \$t4, TABLA2(\$t3)
.code			daddi \$t3, \$t3, 8 <i>nueva posición en Tabla 2</i>
ld \$t0, CANT(\$zero) <i>carga contador</i>		FUERA:	daddi \$t2, \$t2, 8
dadd \$t1, \$zero, \$zero <i>T1=RES</i>			daddi \$t0, \$t0, -1
daddi \$t2, \$t0, 0 <i>t2</i>			bnez \$t0, LOOP
dadd \$t3, \$zero, \$zero			sd \$t1, RES(\$t0)
			halt

3. Describa brevemente cómo se generan los atascos del tipo Read after Write (RAW). (5 pts)

Los atascos RAW se generan cuando una instrucción intenta acceder a un valor que no se ha terminado de escribir en una instrucción previa. Ej. un daddi \$t1, \$t1, 1 seguido de beqz \$t1, \$t2, algo.

4. Escribir un programa que use la información de 5 pares de números guardados en la variable pares, y dibuje en la pantalla gráfica 5 puntos naranjas de acuerdo a la diferencia en valor absoluto de los valores de cada par. (40 pts)

La variable pares contiene todos los valores de los pares, en la primera y segunda posición se guarda el primer par, en posición 3 y 4 el segundo, y así sucesivamente.

El procesamiento de los pares lo debe hacer la subrutina GRAFICAR que recibe por referencia el arreglo de pares y la cantidad. Para cada par de números leídos calcula la diferencia en valor absoluto y dibuja en la pantalla gráfica del simulador un punto naranja en la posición (diferencia, diferencia). El cálculo de la diferencia se realiza en la subrutina DIFERENCIA que recibe los dos números del par y devuelve la diferencia en valor absoluto. Recuerde que la diferencia en valor absoluto se obtiene restando el número mayor al menor.

En todo el ejercicio se debe usar la convención para nombrar a los registros.

```
.data
CONTROL: .word 0x10000
DATA: .word 0x10008

naranja: .byte 255, 128, 0, 0
pares: .word 10, 15, 24, 12, 30, 31, 42, 21, 33, 33
```

data

```
CONTROL .word 0x10000
DATA .word 0x10008
NARANJA .byte 255, 128, 0, 0
PARES .word 10, 15, 24, 12, 30, 31, 42, 21, 33, 33
```

code

```
ld $s0, CONTROL(0) ✓
ld $s1, DATA(0) ✓
ld $s2, NARANJA(0) ✓
sd $s2, 0($s1) # mando el color a Data
daddi $sp, $sp, 0x400 # inicio SP
daddi $a0, $zero, 5 # paso cont de pares
daddi $a1, $0, PARES # paso direccion de vector
jal graficar ✓
halt
```

diferencia: slt \$v0, \$a1, \$a2 ✓

bnez \$v0, menor ✓

dsub \$v0, \$a1, \$a2 ✓

jolver ✓

menor: dsub \$v0, \$a2, \$a1 ✓

wolver: jr \$ra ✓

graficar:

```
daddi $sp, $sp, -24 ✓
sd $a0, $sp(0) ✓
sd $a1, $sp(4) ✓
sd $ra, $sp(8) ✓
```

salvo a0, a1, \$ra

```
dadd $t0, $a0, $0 # T0 = contador de loops
```

```
dadd $t1, $a1, $0 # t1 = direccion de vector
```

```
daddi $t2, $0, 0 # posicion aux
```

lazo: ld \$a0, \$t1(\$t2) # 1º valor del par en a0

```
daddi $t2, $t2, 8 ✓
```

```
ld $a1, $t1($t2) # 2º valor en a1
```

```
daddi $t2, $t2, 8 ✓
```

jal diferencia ✓

```
sd $v0, 4($s1) # mando valor a eje X
```

```
sd $v0, 5($s1) # mando " " " " Y
```

```
daddi $t3, $0, 5 # deberia haberlo inicializado afuera
```

```
sd $t3, 0($s0) ✓
```

```
daddi $t0, $t0, -1 # contador
```

```
bnez $t0, lazo ✓
```

```
ld $ra, $sp(16) ✓
```

```
ld $a1, $sp(8) ✓
```

```
ld $a0, $sp(0) ✓
```

```
daddi $sp, $sp, 24 ✓
```

rescato de la pila

```
jr $ra ✓
```

1. Indicar cuantos atascos se producen del tipo RAW y BTS ejecutando el programa, BTB y Delay Slot están deshabilitados (40 pts)

.data	
VEC1: .word 3, 4, 5	
VEC2: .word 0	
.code	
	loop: ld \$t2, VEC1(\$t0)
	daddi \$t1, \$t1, -1
	sd \$t2, VEC2(\$t0)
	daddi \$t0, \$t0, 8
	bnez \$t1, loop
	halt
	daddi \$t0, \$t0, 0
	daddi \$t1, \$t0, 3

Ejercicio	Puntos
1	15
2	15
3	0
4	0
Total	30

Con Forwarding deshabilitado: RAW: 4 (5 pts) BTS: 2 (5 pts) CPI: 28/18 (10 pts)
 Con Forwarding habilitado: RAW: 0 (5 pts) BTS: 2 (5 pts) CPI: 4/3 (10 pts)

2. El siguiente programa recorre un arreglo de números (TABLA1) y genera un segundo arreglo (TABLA2) con los números que sean pares. Además guarda en RES la cantidad de elementos guardados en el segundo arreglo. Completar instrucciones faltantes. (5 pts c/respuesta)

.data	
TABLA1: .word 11, 2, 6, 7, 4, 21, 10, 4	
CANT: .word 8	
RES: .word 0	
TABLA2: .word 0	
.code	
ld \$t0, CANT(\$zero)	
dadd \$t1, \$zero, \$zero	
<u>dadd \$t2, 10, 10</u>	
dadd \$t3, \$zero, \$zero	
	LOOP: ld \$t4, TABLA1(\$t2)
	andi \$t5, \$t4, 1
	bnez \$t5, FUERA
	daddi \$t1, \$t1, 1
	<u>sd \$t4, TABLA2(\$t3)</u>
	daddi \$t3, \$t3, 8
	FUERA: daddi \$t2, \$t2, 8
	daddi \$t0, \$t0, -1
	bnez \$t0, LOOP
	<u>sd RES, \$t1(\$t0)</u>
	halt

3. Describa brevemente cómo se generan los atascos del tipo Write after Write (WAW). (5 pts)

ES UN ATASCO DE TIPO ESTRUCTURAL; OCURRE EN LA ETAPA WB, CUANDO SE DESA ESCRIBIR SOBRE UN DATO QUE *

4. Escribir un programa que use la información de 4 pares de números guardados en la variable pares, y dibuje en la pantalla gráfica 4 puntos grises de acuerdo al promedio de valores de cada par. (40 pts)

La variable pares contiene todos los valores de los pares, en la primera y segunda posición se guarda el primer par, en posición 3 y 4 el segundo, y así sucesivamente.

El procesamiento de los pares lo debe hacer la subrutina GRAFICAR que recibe por referencia el arreglo de pares y su cantidad. Para cada par de números leídos calcula su promedio y dibuja en la pantalla gráfica del simulador un punto gris en la posición (promedio, promedio). El cálculo del promedio se realiza en una subrutina que recibe los dos números del par y devuelve el promedio. Recuerde que el promedio de un par es la suma de los dos números dividido 2.

En todo el ejercicio se debe usar la convención para nombrar a los registros.

```
.data
CONTROL: .word 0x10000
DATA: .word 0x10008

gris: .byte 128, 128, 128, 0
pares: .word 8, 16, 20, 24, 3, 5, 15, 31
```

ALCOBATOS (90)

GF

Arquitectura de Computadoras
Apellido: _____

Parcial 2- Fecha 1 - 15/11/2024
Nombre: _____

TEMA B
DNI: _____

1. Indicar cuantos atascos se producen del tipo RAW y BTS ejecutando el programa, BTB y Delay Slot están deshabilitados (40 pts)

.data	loop: ld \$t2, VEC1(\$t0)
VEC1: .word 5, 10	daddi \$t1, \$t1, -1
VEC2: .word 0	sd \$t2, VEC2(\$t0)
.code	daddi \$t0, \$t0, 8
daddi \$t0, \$0, 0	bnez \$t1, loop
daddi \$t1, \$0, 2	halt

Ejercicio	Puntos
1	40
2	15
3	5
4	30
Total	90

Con Forwarding deshabilitado: RAW: 3 (5 pts) BTS: 1 (5 pts) CPI: 21/13 (10 pts)
Con Forwarding habilitado: RAW: 0 (5 pts) BTS: 1 (5 pts) CPI: 18/13 (10 pts)

2. El siguiente programa recorre un arreglo de números (TABLA1) y genera un segundo arreglo (TABLA2) con los números que sean impares. Además guarda en RES la cantidad de elementos guardados en el segundo arreglo. Completar instrucciones faltantes. (5 pts c/respuesta)

.data	LOOP: ld \$t4, TABLA1(\$t2)
TABLA1: .word 11, 2, 6, 7, 4, 21, 10, 4	andi \$t5, \$t4, 1
CANT: .word 8	beqz \$t5, FUERA
RES: .word 0	daddi \$t1, \$t1, 1
TABLA2: .word 0	sd \$t4, TABLA2(\$t3)
.code	daddi \$t3, \$t3, 8 <i>nueva posición en Tabla 2</i>
ld \$t0, CANT(\$zero) <i>carga contador</i>	daddi \$t2, \$t2, 8
dadd \$t1, \$zero, \$zero <i>T1=RES</i>	daddi \$t0, \$t0, -1
daddi \$t2, \$t0, 0 <i>t2</i>	bnez \$t0, LOOP
dadd \$t3, \$zero, \$zero	sd \$t1, RES(\$t0)
	halt

3. Describa brevemente cómo se generan los atascos del tipo Read after Write (RAW). (5 pts)

Los atascos RAW se generan cuando una instrucción intenta acceder a un valor que no se ha terminado de escribir en una instrucción previa. Ej. un daddi \$t1, \$t1, 1 seguido de beqz \$t1, \$t2, algo.

4. Escribir un programa que use la información de 5 pares de números guardados en la variable pares, y dibuje en la pantalla gráfica 5 puntos naranjas de acuerdo a la diferencia en valor absoluto de los valores de cada par. (40 pts)

La variable pares contiene todos los valores de los pares, en la primera y segunda posición se guarda el primer par, en posición 3 y 4 el segundo, y así sucesivamente.

El procesamiento de los pares lo debe hacer la subrutina GRAFICAR que recibe por referencia el arreglo de pares y la cantidad. Para cada par de números leídos calcula la diferencia en valor absoluto y dibuja en la pantalla gráfica del simulador un punto naranja en la posición (diferencia, diferencia). El cálculo de la diferencia se realiza en la subrutina DIFERENCIA que recibe los dos números del par y devuelve la diferencia en valor absoluto. Recuerde que la diferencia en valor absoluto se obtiene restando el número mayor al menor.

En todo el ejercicio se debe usar la convención para nombrar a los registros.

```
.data
CONTROL: .word 0x10000
DATA: .word 0x10008

naranja: .byte 255, 128, 0, 0
pares: .word 10, 15, 24, 12, 30, 31, 42, 21, 33, 33
```

Arquitectura de Computadoras
Apellido: CASA MA YOV

Parcial 2- Fecha 1 - 15/11/2024 - A
Nombre: IRMA SEVENE

Aula: 9
DNI: [REDACTED]

1. Indicar cuantos atascos se producen del tipo RAW y BTS ejecutando el programa, BTB y Delay Slot están deshabilitados (40 pts)

.data NUM: .word 3	loop: daddi \$t2, \$0, 2 dadd \$t0, \$t0, \$t1 daddi \$t2, \$t2, -1 bnez \$t2, loop halt
.code ld \$t1, NUM(\$0) daddi \$t0, \$0, 0	

Ejercicio	Puntos
1	35
2	15
3	5
4	40
Total	95

Forwarding deshabilitado: RAW: 4 (5p) BTS: 1 (5p) CPI: 19 / 10 (10p)
Forwarding habilitado: RAW: 2 (5p) BTS: 1 (5p) CPI: 17 / 10 (10p)

2. El siguiente programa recorre un arreglo de números (TABLA1) y genera un segundo arreglo (TABLA2) con los números que sean menores que LIMITE. Además guarda en RES la cantidad de elementos guardados en el segundo arreglo. Completar instrucciones faltantes. (5 pts c/respuesta)

.data TABLA1: .word 1, 2, 6, 3, 4, 5, 10, 11 CANT: .word 8 LIMITE: .word 3 RES: .word 0 TABLA2: .word 0	.code ld \$t0, CANT(\$zero) ld \$t1, LIMITE(\$zero) dadd \$t2, \$zero, \$zero dadd \$t3, \$zero, \$zero
	LOOP: ld \$t5, TABLA1(\$t3) slt \$t6, \$t5, \$t1 DEB2 \$t6, FUERA daddi \$t2, \$t2, 1 sd \$t5, TABLA2(\$t4) DADDI \$t4, \$t4, 8 FUERA: daddi \$t3, \$t3, 8 lwd \$t0, \$t0, -1 bnez \$t0, LOOP sd \$t2, RES(\$zero) halt

3. Describa brevemente cómo se generan los atascos del tipo Branch Token Stall (BTS). (5 pts)

ESTOS SUCEDEN CUANDO EL SALTO DICE QUE NO SALTE, Y TENIA QUE SALTAR

4. Escribir un programa que utilice 7 pares de números guardados en la variable datos, y dibujar en la pantalla gráfica puntos azules con los pares de números como coordenadas X e Y de cada punto, descartando los puntos que se encuentren fuera del rango de la pantalla gráfica del simulador. (40 pts)

La variable datos contiene todos los valores de los pares, en la primera y segunda posición se guarda el primer par, en posición 3 y 4 el segundo par, y así sucesivamente.

Para realizar el gráfico escribir la subrutina GRAFICAR que recibe por referencia la dirección del arreglo que tiene los pares cargados y la cantidad de pares. Para cada par de números hay que verificar que ambos números están entre 0 y 49 (incluidos). En caso de estar en rango, pintar el punto de color azul, en caso contrario no hacer nada. Dicha verificación la realiza una subrutina llamada EN_RANGO. La subrutina EN_RANGO recibe el número a verificar y devuelve 1 si está en el rango y 0 si no lo está. La subrutina no dibuja, solo verifica que un número esté en rango.

En todo el ejercicio se debe usar la conversión para relacionar a los registros.

.data
CONTROL: .word 0x10000
DATA: .word 0x10008

azul: .byte 0, 0, 255, 0
datos: .word 35, 20, 70, 20, 1, 1, -10, 5, 20, 30, -3, 10, 12, 20

APROBADO (95)

JLSP

Arquitectura de Computadoras

Apellido: Vazquez

Parcial 2- Fecha 1-15/11/2024

Nombre: Diana Carolina

TEMA A

DNI: 4444007

1. Indicar cuantos atascos se producen del tipo RAW y BTS ejecutando el programa. BTS y Delay Slot están deshabilitados (40 pts)

(40)

```
.data
DATO: .word 3

.code
ld $t1, DATO($0)
daddi $t0, $0, 0

loop:
daddi $t2, $0, 3
dadd $t0, $t0, $t1
daddi $t2, $t2, -1
bnez $t2, loop
halt
```

Con Forwarding deshabilitado: RAW: 3 (5 pts) BTS: 2 (5 pts) CPI: 2.2 (10 pts)
 Con Forwarding habilitado: RAW: 3 (5 pts) BTS: 2 (5 pts) CPI: 2.2 (10 pts)

Pregunta	Puntos
1	40
2	15
3	5
4	35
Total	95

2. El siguiente programa recorre un arreglo de números (TABLA1) y genera un segundo arreglo (TABLA2) con los números que sean mayores que LIMITE. Además guarda en RES la cantidad de elementos guardados en el segundo arreglo. Completar instrucciones faltantes. (5 pts c/respuesta)

(15)

```
.data
TABLA1: .word 1,2,6,3,4,5,10,11
CANT: .word 8
LIMITE: .word 3
RES: .word 0
TABLA2: .word 0

.code
ld $t0, CANT($zero)
ld $t1, LIMITE($zero)
dadd $t2, $zero, $zero
dadd $t3, $zero, $zero

LOOP:
ld $t5, TABLA1($t2)
slt $t6, $t5, $t1
beqz $t6, FUERA
daddi $t2, $t2, 1
sd $t5, TABLA2($t4)
daddi $t4, $t4, 8
daddi $t3, $t3, 8
daddi $t0, $t0, -1
bnez $t0, LOOP
sd $t2, RES($zero)
halt

FUERA:
daddi $t2, $t2, 8
```

dadd \$t3 \$zero, \$zero

(5)

3. Describa brevemente cómo se generan los atascos del tipo Structural Stall (STR). (5 pts)

Los atascos estructurales se generan cuando dos instrucciones al mismo ciclo/hardware, dividen o usan el mismo recurso.

(35)

4. Escribir un programa que utilice 6 pares de números guardados en la variable datos, y dibujar en la pantalla gráfica puntos púrpuras con los pares de números como coordenadas X e Y de cada punto, descartando los puntos que se encuentren fuera del rango de la pantalla gráfica del simulador. (40 pts)

La variable datos contiene todos los valores de los pares, en la primera y segunda posición se guarda el primer par, en posición 3 y 4 el segundo par, y así sucesivamente.

Para realizar el gráfico escribir la subrutina GRAFICAR que recibe por referencia la dirección del arreglo que tiene los pares cargados y la cantidad de pares. Para cada par de números hay que verificar que ambos números están entre 0 y 49 (incluidos). En caso de estar en rango, pintar el punto de color púrpura, en caso contrario no hacer nada. Dicha verificación la realiza una subrutina llamada EN_RANGO. La subrutina EN_RANGO recibe el número a verificar y devuelve 1 si está en el rango y 0 si no lo está. La subrutina no dibuja, solo verifica que un número está en rango.

En todo el ejercicio se debe usar la convención para nombrar a los registros.

```
.data
CONTROL: .word 0x10000
DATA: .word 0x10008

purgura: .byte 128, 0, 128, 0
datos: .word 1, 50, 35, 16, 15, 15, -1, 0, 49, 49, 3, 33
```

(*) la primer instrucción que se haya corrido.

• Code

```
daddi $a0, $0, datos
daddi $a2, $0, 6
daddi $sp, $0, 0x400
Jal Graficos
halt
```

Graficos:

REG GRA

```
daddi $fp, $sp, -8
ld $ra, 0($sp)
```

```
lw $s0, CONTROL($0)
lw $s1, DATA($0)
lb $s2, PURPURA($0)
sb $s2, 0($s1)
```

loop: lw \$a0, 0(\$a0) caso primer n°

JAL EN-RANGO

bgez \$v0, siguientePor

dadd \$s3, \$0, \$v0

daddi \$a0, \$0, 8

JAL EN-RANGO

bgez \$v0, siguienteElemento

sd \$s3, 5(\$s1)

sd \$a1, 4(\$s1)

daddi \$t2, \$0, 5

sd \$t2, 0(\$s0)

siguientePor: daddi \$a0, \$0, 16

J fin

fin: daddi \$a2, \$0, -1

bnez \$a2, loop

wp \$ra

sd \$ra, 0(\$sp)

daddi \$sp, \$sp, 8

JR \$RA

Proceder en al el n°, disminuir en 10

EN-RANGO

~~XXXXXXXXXXXX~~

si \$t1, \$a1, 0

bgez \$t1, no-ra

si \$t1, \$a1, 0

bgez \$t1, no-ra

daddi \$v0, \$0, 1

J fin

no-ra: daddi \$v0, \$0, 0

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA

fin: JR \$RA